



Project Title: How HIV death rates changed over time globally?

Name: Kashir Hasnain

Mode: ☒ 6 ECTS

Representations

(LO1) Understand and apply Knowledge Graph Embeddings	☐ I showed basic proficiency ☐ exceeded basic proficiency
Used Logic Based (SPARQL) representation as stated in LO2.	Nope

(LO2) Understand and apply logical knowledge in KGs	☒ I showed basic proficiency ☐ exceeded basic proficiency
Designed Queries over the Knowledge Graph: 1. a basic pattern query 2. An Aggregation query 3. a recursive property-path query 4. CONSTRUCT-style rule that materializes new has Trend edges.	Section 3.1 (p.10-11)

(LO3) Understand and apply Graph Neural Networks	☐ I showed basic proficiency ☐ exceeded basic proficiency
LO2 and LO6, Implemented a graph structure via SPARQL Queries.	N/A

(LO4) Compare different Knowledge Graph data models from the database, semantic web, machine learning and data science communities.	☐ I showed basic proficiency ☒ exceeded basic proficiency
---	---



Compared the RDF Graph Data Model used here against the original CSV Model labeled Property Graph model discussing trade-offs using for this use case.	Section 4.2 (p.14)
--	--------------------

Systems

(LO5) Design and implement architectures of a Knowledge Graph	☒ I showed basic proficiency ☐ exceeded basic proficiency
Populate and Query the Knowledge Graph (Python, rdflib, Turtle and SPARQL Engine).	Section 2.2 (p.8-9)

(LO6) Describe and apply scalable reasoning methods in Knowledge Graphs	☒ I showed basic proficiency ☐ exceeded basic proficiency
Discussed SPARQL Based reasoning approach scales along volume.	Section 3.3 (p.11-12)

(LO7) Apply a system to create a Knowledge Graph	☐ I showed basic proficiency ☒ exceeded basic proficiency
1. I designed an ontology (Country, WHORegion, Year, Observation classes and locatedIn/ofCountry/inYear/count properties) 2. I implemented an ETL pipeline that populated the KG (6,560+ triples) from six WHO/UNAIDS CSV files.	Section 2 & 2.3 (p.8-9)

(LO8) Apply a system to evolve a Knowledge Graph	☐ I showed basic proficiency ☐ exceeded basic proficiency
--	--



Not included in this Knowledge Base Project.	Nope
--	------



Applications

(LO9) Describe and design real-world applications of Knowledge Graphs	☒ I showed basic proficiency ☐ exceeded basic proficiency
Global Public Health Surveillance of the HIV/AIDS epidemic and treatment response.	Section 1 (p.8)
(LO10) Describe financial Knowledge Graph applications	☐ I showed basic proficiency ☐ exceeded basic proficiency
Not included.	Nope
(LO11) Apply a system to provide services through a Knowledge Graph	☒ I showed basic proficiency ☐ exceeded basic proficiency
service the KG provides -- a class of temporal/regional HIV death-trend queries and country trend classifications. -- implemented it end-to-end.	Section 1 & 4.1 (p.8, 12-13)
(LO12) Describe the connections between Knowledge Graphs (KGs), Machine Learning (ML) and Artificial Intelligence (AI)	☒ I showed basic proficiency ☐ exceeded basic proficiency
I reflected on how this logic-based KG relates to the ML-based representations (embeddings, GNNs) that were deliberately out of scope, and where they could complement it.	Section 4.3 (p.14-15)



Additional Information

HAS NO EFFECT ON MARKING!

(please fill it out honestly, even if it is less than what is suggested in the ECTS breakdown – you are not judged on time spent!)

How many hours did you spend on your mini-project ?	74 hours
--	----------

* please exclude any hours you spent on parts reused from other courses

How many hours did you spend on your portfolio document preparation (this PDF) ?	4 hours
---	---------

* please exclude any hours you spent on parts reused from other courses

Declaration

I have marked all parts generated by Generative AI (e.g., ChatGPT) or otherwise copied from other sources and given any prompt I used either in a footnote or in an appendix making clear which parts are generated by which prompts or similar.	☐ I confirm this
--	---

Please indicate if you have used generative AI parts of the mini-project	☒ I used it for roughly 5% of the mini-project
I fixed the minor bugs in the code at the end of the Project as I am new to matplotlib and the plotting. I learned it first then implement it.	

Please indicate if you have used generative AI parts of this document	☐ I used it for roughly <XX>% of this document
Not used in the document.	



Introduction:

Domain Knowledge (LO9): HIV/AIDS are increasing in today world, as stated from the World Health Organization website estimated 41.0 million cases are discovered in the 2025 and most of the cases are from African Countries but other cases are from rest of the world. HIV is a virus which disturbs immune system and in the immune system it will target the white blood cells which are less in the body and necessary for the immune system otherwise it will infection and cancer. Representing this as a graph makes it possible to traverse from a single country, up to its region, across years, and across indicators, in a single query — something that is considerably more awkward in a purely tabular (CSV/relational) representation, where every such traversal requires a fresh joins.

Knowledge Graph Construction:

Dataset:

The KG is populated from the cleaned “HIV/AIDS Dataset” published on Kaggle (imdevskp/hiv-aids-dataset, <https://www.kaggle.com/datasets/imdevskp/hiv-aids-dataset>). The dataset contains six CSV files; this project uses three of them directly and a fourth as supporting context:

- **no_of_deaths_by_country_clean.csv** — Country, Year (2000/2010/2018), Count (median[min–max]), Count_median, Count_min, Count_max, WHO Region. This is the core dataset (511 rows).
- **no_of_people_living_with_hiv_by_country_clean.csv** — same schema, used as the prevalence indicator.
- **art_coverage_by_country_clean.csv** — a single-snapshot indicator (Reported number receiving ART, Estimated ART coverage %, WHO Region), used to relate treatment access to the observed death-rate trend.

Architecture (LO5):

The knowledge graph is built in python using rdflib which is an in-memory RDF Store with the build in SPARQL Engine and saved to disk as a turtle file. The whole project is run through the single python script with no extra tools to install, configure or maintain it. That’s why I used rdflib because it also helps with the scalability.

Knowledge Graph Construction:

The ontology (namespace hiv: = <http://example.org/hiv-kg#>) defines four core classes and reuses them across all three source datasets so that data originally split across separate CSV files becomes a single connected graph:

- hiv:Country, hiv:WHORegion, hiv:Year — the shared dimensional entities.
- hiv:DeathObservation, hiv:PrevalenceObservation, hiv:ARTCoverageObservation — indicator observations, each linked to a Country (and, where time-stamped, a Year) via hiv:ofCountry / hiv:inYear, carrying the value via hiv:count (plus hiv:countMin / hiv:countMax for the reported uncertainty interval).



- Object properties `hiv:locatedIn` (Country → WHORegion) and `hiv:partOf` (WHORegion → `hiv:Global`) form an explicit containment hierarchy used by the recursive query in Section 3.1.

Three concrete examples of how nodes/edges were constructed from the raw CSV rows:

- Row: Kenya, 2018, "25000[21000–31000]", 25000, 21000, 31000, Africa
-> `hiv:country_Kenya a hiv:Country ; rdfs:label "Kenya" ; hiv:locatedIn hiv:region_Africa .`
-> `hiv:deathobs_Kenya_2018 a hiv:DeathObservation ;`
 `hiv:ofCountry hiv:country_Kenya ; hiv:inYear hiv:year_2018 ;`
 `hiv:count "25000"^^xsd:double ; hiv:countMin "21000"^^xsd:double ; hiv:countMax "31000"^^xsd:double .`
-> `hiv:country_Kenya hiv:hasDeathObservation hiv:deathobs_Kenya_2018 .`
- Row: WHO Region = "Africa" (encountered while parsing any of the country rows)
-> `hiv:region_Africa a hiv:WHORegion ; rdfs:label "Africa" ; hiv:partOf hiv:Global .`
- Row (`art_coverage_by_country_clean.csv`): Kenya, ART coverage median = 75%
-> `hiv:artobs_Kenya a hiv:ARTCoverageObservation ;`
 `hiv:ofCountry hiv:country_Kenya ; hiv:count "75"^^xsd:double .`

Rows with “na” / “Nodata” values (e.g. several small territories with no reported ART coverage) are skipped rather than populated with a placeholder value, so that the absence of a triple faithfully represents the absence of data rather than an artificial zero.

Populating all three datasets produced 141 `hiv:Country` nodes, 6 `hiv:WHORegion` nodes, 400 death observations, 553 prevalence observations and 136 ART-coverage observations — 6,560 triples in total.

Logic Based Representation / Queries:

The logic-based representation consists of five SPARQL queries/rules that operate directly over the KG built in Section 2.

Q1 — basic pattern query (all reported death counts for a given country):

```
SELECT ?year ?count WHERE {  
  ?c a hiv:Country ; rdfs:label "Kenya" .  
  ?obs a hiv:DeathObservation ; hiv:ofCountry ?c ; hiv:inYear ?y ; hiv:count ?count .  
  ?y hiv:yearValue ?year .  
}  
ORDER BY ?year  
-> 2000: 120000 2010: 56000 2018: 25000
```

Q2 — aggregation query (total & mean deaths per WHO Region, per year):

```
SELECT ?regionLabel ?year (SUM(?count) AS ?totalDeaths) (AVG(?count) AS ?meanDeaths)  
WHERE {  
  ?obs a hiv:DeathObservation ; hiv:ofCountry ?c ; hiv:inYear ?y ; hiv:count ?count .  
  ?c hiv:locatedIn ?region . ?region rdfs:label ?regionLabel .  
  ?y hiv:yearValue ?year .  
}  
GROUP BY ?regionLabel ?year ORDER BY ?regionLabel ?year
```



Q3 — recursive exploration (property path over locatedIn|partOf, following an arbitrary number of edges up the Country → WHORegion → Global containment hierarchy):

```
SELECT ?countryLabel ?ancestor WHERE {  
  ?c a hiv:Country ; rdfs:label "Rwanda" .  
  ?c (hiv:locatedIn|hiv:partOf)+ ?anc .  
  OPTIONAL { ?anc rdfs:label ?lbl }  
  BIND(COALESCE(?lbl, STR(?anc)) AS ?ancestor)  
}  
-> Rwanda -> Africa  
-> Rwanda -> Global
```

Q4 — a rule that creates new nodes/edges in the graph. Each country's death counts for 2000 and 2018 are retrieved and compared; the percentage change is used to fire a classification rule (StronglyImproved if $\leq -30\%$, Improved if $< 0\%$, Stable if $0-30\%$, Worsened if $> 30\%$), and the result is written back into the graph as a new triple:

```
IF pct_change(deaths[2000], deaths[2018]) <= -30% THEN  
  INSERT { ?country hiv:hasTrend hiv:StronglyImproved }  
# ... analogous rules for Improved / Stable / Worsened  
  
# example fired instances:  
hiv:country_Rwanda hiv:hasTrend hiv:StronglyImproved . # -83.9%  
hiv:country_Pakistan hiv:hasTrend hiv:Worsened . # +6300.0%
```

Running this rule over all 131 countries with data for both years added 131 new hiv:hasTrend triples to the graph (verified by re-querying the graph afterwards), turning an implicit numerical fact spread across two separate observation nodes into an explicit, directly queryable graph edge.

Q5 — a follow-up query used to relate the newly created hiv:hasTrend edges to the ART-coverage indicator (used narratively in Section 4.1):

```
SELECT (AVG(?art) AS ?meanART) WHERE {  
  ?c hiv:hasTrend hiv:StronglyImproved .  
  ?a a hiv:ARTCoverageObservation ; hiv:ofCountry ?c ; hiv:count ?art .  
}  
-> mean ART coverage, StronglyImproved countries: 66.3% (n=43)  
-> mean ART coverage, Worsened countries: 40.0% (n=36)
```

Output:

Query Q2, which aggregates deaths by WHO region, makes it clear that the global drop in HIV deaths between 2000 and 2018 is really an African story more than anything else. Reported deaths in the Africa region fell from around 1.07 million in 2000 to about 463,000 in 2018 — a 57% reduction. The other five regions barely moved by comparison, and Eastern Mediterranean actually went up over the same period.

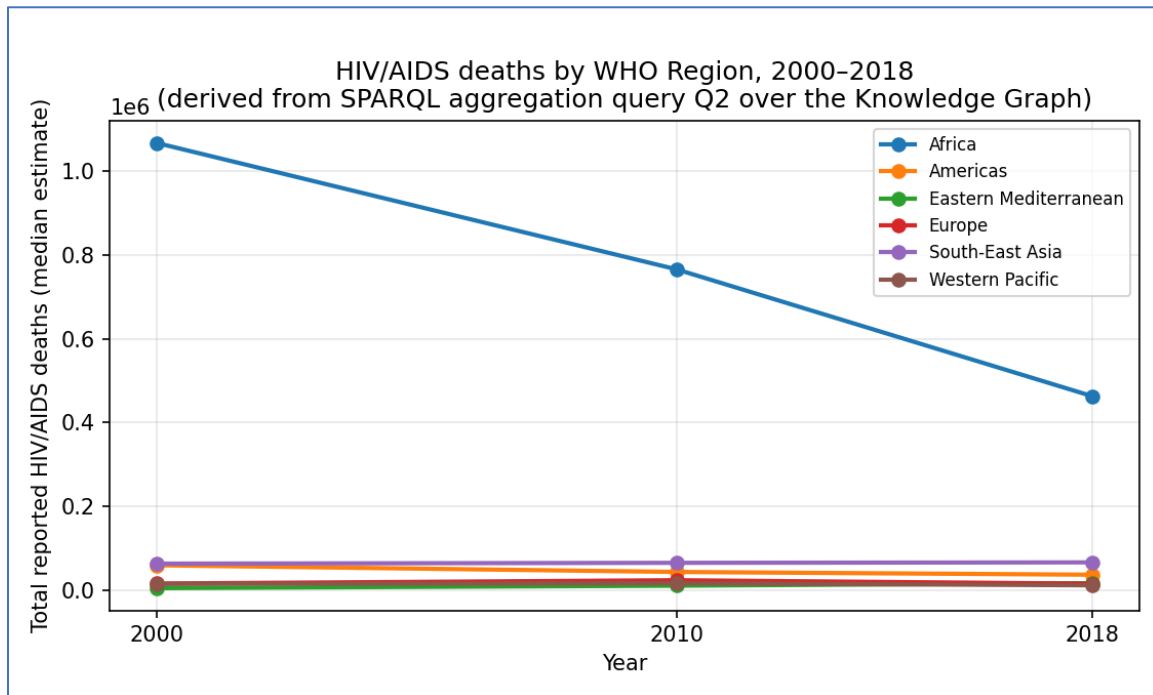


Figure 1. Total HIV/AIDS deaths by WHO Region, 2000–2018 (Q2 results).

The trend-classification rule (Q4) makes this visible at country level: of the 131 countries with usable data for both 2000 and 2018, 43 were classified StronglyImproved ($\geq 30\%$ reduction), 14 Improved, 38 Stable, and 36 Worsened

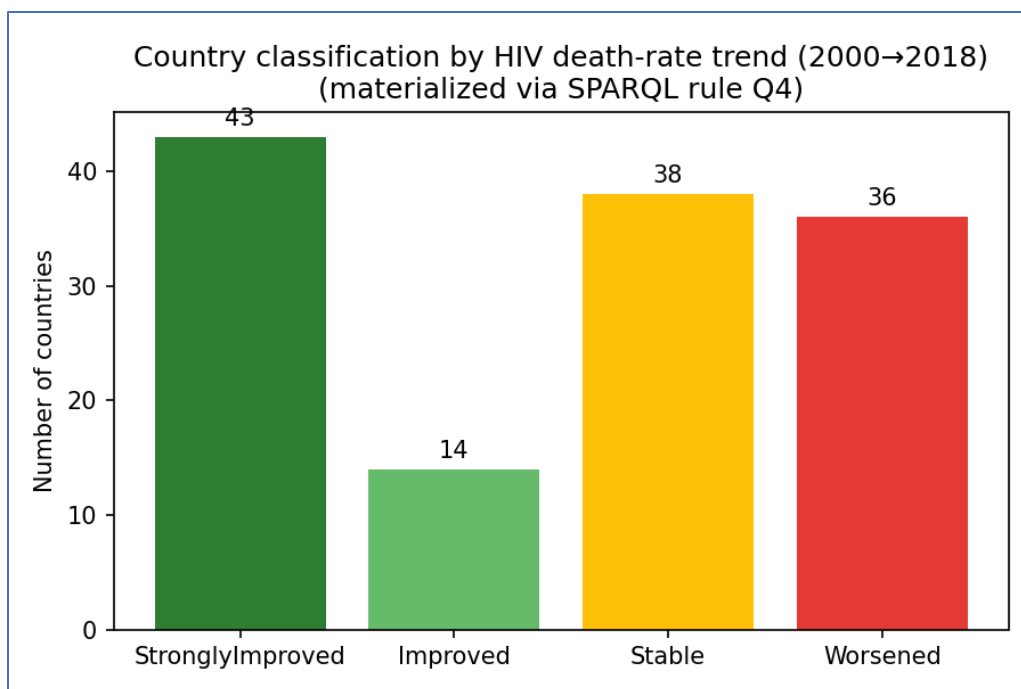


Figure 2. Countries by HIV death-rate trend class, 2000→2018 (Q4 results).

The trend-classification rule (Q4) makes this visible at country level: of the 131 countries with usable data for both 2000 and 2018, 43 were classified StronglyImproved ($\geq 30\%$ reduction), 14 Improved, 38 Stable, and 36 Worsened.



Reflection on the data model and connections to ML/AI (LO4, LO12)

The data started life as six CSVs — a tabular model. I chose to represent it instead as an RDF graph queried with SPARQL, mainly because relationships like country–region and country–year become edges, rather than columns you have to join every time you ask a question. A query like "which countries in region X had over Y% ART coverage in the year their deaths fell fastest" needs several joins in pandas or SQL; over the graph (Q2+Q4+Q5 composed) it's a handful of triple patterns. That convenience isn't free, though it took an up-front ontology-design step (Section 2.3) before I could run any query at all, which loading a CSV straight into pandas doesn't require. I went with RDF because treating each observation as its own node made it easy to attach an uncertainty interval, and later a `hiv:hasTrend` edge, without touching anything I'd already built or changing the schema.

I stuck to the logic-based half of the KG toolbox here — an explicit ontology plus SPARQL rules — and left embeddings and GNNs out of scope. That's most obvious in the "na" rows I had to skip in Section 2.3: Albania, for example, simply has no ART-coverage figure reported. A trained embedding could plausibly guess one by comparing Albania to structurally similar countries and filling the gap. The rules I used instead (Q4) don't do that — they're fully traceable back to the source data, but they just go quiet wherever the facts aren't there.

I don't think of these as competing approaches, though. An embedding could fill in something like Albania's missing figure, and the same Q4 rule would then run over the completed graph without any changes, since all it cares about is whether a `hiv:count` triple exists not whether it came from a CSV row or a prediction. That's really the point of connection between the course's three representation families i.e. logical, embedding-based, and GNN-based: they all sit on top of the same graph, and could, in principle, be chained together in one pipeline rather than treated as separate choices.